

## PAPER

# A Highly Efficient Neural Distinguisher Framework for IoT-Friendly Lightweight SPN Block Ciphers

Jiashuo LIU<sup>†</sup>, Manman LI<sup>†a)</sup>, Jiongjiong REN<sup>†</sup>, and Shaozhen CHEN<sup>†</sup>, *Nonmembers*

**SUMMARY** In the past few years, research on lightweight block ciphers as security ciphers in the Internet of Things (IoT) has attracted considerable attention in cryptography. In this paper, we present an improved framework for neural distinguishers in lightweight SPN block ciphers suitable for IoT, focusing on two aspects: training data format and neural network structure. First, we analyze the nature of the SPN round function, then divide it into three cases to apply data augmentation. Second, we generate training data samples using three dimensions and construct neural networks using two-dimensional convolution. Finally, we validate the advantages of the improved framework on the SKINNY family and MIDORI family with higher accuracy and achieve a breakthrough in the number of rounds.

**key words:** deep learning, SPN block ciphers, differential cryptanalysis, SKINNY, MIDORI

## 1. Introduction

The Internet of Things (IoT) is advancing at a breathtaking pace. It has penetrated various aspects of life, from smart homes to industrial automation, bringing remarkable convenience and efficiency. However, with its rapid expansion, numerous security risks emerge, such as data breaches and device hijacking. Thus, safeguarding IoT security is of utmost importance to protect privacy and to ensure smooth social operations.

Lightweight block ciphers have emerged as a promising solution for IoT security. Their low-power, low-memory-footprint, and fast-encryption characteristics make them highly compatible with resource-constrained IoT devices. The Substitution Permutation Networks (SPN) structure is a common design scheme which is the result of the seminal work of Shannon [1]. The key operations of the SPN structure include substitution and permutation. Substitution introduces non-linearity by replacing bits in a data block with S-boxes. Meanwhile, the permutation operation rearranges the bits in the data block to aid diffusion. Its relatively simple design is conducive to implementation in hardware-limited IoT devices. There are many well designed lightweight SPN block ciphers such as SKINNY [2], MIDORI [3], PRESENT [4].

To guarantee the secure utilization of lightweight block ciphers in the IoT environment, conducting a comprehensive security analysis is of utmost necessity. Up to the present day, multiple analysis methods have surfaced for block ci-

phers, including differential cryptanalysis [5], linear cryptanalysis [6], integral cryptanalysis [7], zero-correlation linear cryptanalysis [8], and others. Recently, machine learning, particularly deep learning techniques, has become increasingly popular among cryptanalysis practitioners as a valuable tool.

At CRYPTO 2019, Gohr [9] introduced differential-neural cryptanalysis, a technique that integrates differential analysis with deep learning. Using deep residual neural networks (ResNet), Gohr trained neural distinguishers for Speck32/64. If the accuracy of a neural distinguisher exceeds 50%, the neural distinguisher can distinguish the target cipher from a pseudorandom permutation. Then Gohr achieved 11-round and 12-round Speck32/64 key retrieval attacks based on 6-round and 7-round neural distinguishers. Gohr's groundbreaking research has ushered in a transformative paradigm in fusing deep learning with cryptanalysis, sparking intensive explorations into neural differential distinguishers.

For neural network architectures serving as differential distinguishers, researchers have explored multiple improvement strategies inspired by computer vision techniques. At ASIACRYPT 2022, Bao et al. [10] proposed neural distinguisher models based on DenseNet and SENet, demonstrating that SENet-based models outperform others in accuracy when targeting 7–9-round Simon32/64. Concurrently, Băcuiet et al. [11] adopted network pruning to lightweight neural distinguishers, reducing training costs without compromising accuracy by compressing network depth.

Regarding training data formats, the unique characteristics of neural distinguisher datasets—abundant samples from cipher encryption but with diffused differential features—have driven innovative solutions. Chen et al. [12] introduced multi-ciphertext pair formats, significantly improving the accuracy and effective rounds of neural distinguishers for SPECK32/64, PRESENT, and DES. Hou et al. [13] further optimized this by converting ciphertext pairs into output differences, enhancing distinguishability for SPECK and Simon families. Ebrahimi et al. [14] constructed partial-bit neural distinguisher models for 6-round SPECK32/64. Lu et al. [15] proposed the first correlation-key neural distinguishers. Hambitzer et al. [16] developed a single-bit prediction ensemble model. Yue et al. [17] proposed a novel input data format and an optimized residual network, constructing a 7-round differential neural distinguisher for Speck32/64 with 97.13% accuracy. Wang et al. [18] refined differential selection via fixed dominant difference screening and MILP tech-

Manuscript received April 3, 2025.

Manuscript revised July 3, 2025.

Manuscript publicized August 21, 2025.

<sup>†</sup>Information Engineering University, Zhengzhou, P.R.China.

a) E-mail: limanman15@163.com

DOI: 10.1587/transinf.2025EDP7070

nology, while Shen et al. [19] expanded classification targets from single to multiple ciphertext differences. Most recently, Hu et al. [20] proposed a novel multi-ciphertext matrix format using penultimate-round leakage differentials, achieving state-of-the-art accuracy for neural distinguishers.

At present, the advancement of neural distinguishers focuses mainly on specialised optimisations for a single cipher algorithms, unfortunately overlooking the formulation of training models tailored to a specific category of structured cipher algorithms. This paper simultaneously delves into the realm of deep learning techniques and cryptanalysis methodologies, presents an improved framework for neural distinguishers in lightweight SPN block ciphers, focusing on training data format and neural network structure. We analyze the nature of the SPN round function, classifying it into three cases for data augmentation, generate training samples using three dimensions, and first construct neural networks with two-dimensional convolution (Conv2D). Notably, this marks a significant departure from prior works (e.g., Gohr [9], Bao et al. [10]), which relied on Conv1D and overlooked the spatial dependencies induced by SPN permutation layers. By treating SPN's matrix-structured state as 2D input, Conv2D captures cross-byte diffusion patterns via kernels—a new approach in differential-neural cryptanalysis. Experimental results validate that Conv2D enhances the 7-round SKINNY-64 distinguisher accuracy from 97.6% to 99.2% while reducing convergence epochs by 80%, demonstrating its superiority in modeling SPN's confusion-diffusion properties. Validated on the SKINNY and MIDORI families, the framework achieves higher accuracy and breakthroughs in the number of rounds, addressing the lack of generic training models for structured cipher algorithms by integrating deep learning and cryptanalysis to create a refined model for SPN block ciphers. More detailed summary of the main contributions can be summarised as follows:

- First, we develop a versatile data augmentation method specifically for SPN structures. Current ciphertext pairs data format ignores the discrepancy between various cipher algorithms. Therefore, we create a new data format that is tailored to the unique structure of SPN block ciphers. Specifically, we classify the SPN block cipher round function into three cases based on component substitution and subkey operation and propose three training data augmentation methods for each.
- Second, we design a better neural network based on the SPN structure. In SPN block ciphers, the use of Conv1D tends to neglect the relationships between bits that are far apart in terms of Euclidean distance. In response, we transform the input data into a three-dimensional format and use two-dimensional convolutions for feature extraction. Two-dimensional convolution can capture more comprehensive structures and patterns in the data.
- Finally, we substantiate the merits of the improved framework through its application to both the SKINNY

**Table 1** Summary of the neural distinguishers for SKINNY and MIDORI family

| Cipher     | Core Net            | Round | Accuracy |
|------------|---------------------|-------|----------|
| SKINNY-64  | Conv1D <sup>1</sup> | 7     | 93.7%    |
|            |                     | 7     | 99.2%    |
|            | Conv2D              | 8     | 54.1%    |
| SKINNY-128 | Conv2D              | 7     | 99.7%    |
|            |                     | 8     | 51.0%    |
| MIDORI-64  | Conv2D              | 3     | 99.3%    |
|            |                     | 4     | 86.1%    |
| MIDORI-128 | Conv2D              | 3     | 97.2%    |
|            |                     | 4     | 72.8%    |

<sup>1</sup> The trail that used to derive this distinguisher comes from Gohr *et al.*'s work [23]. Others are newly found in our work.

and MIDORI families. By training neural distinguishers for SKINNY-64, we achieve the highest accuracy with 7-round neural distinguishers. And for the first time, we get an effective 8-round neural distinguisher. Additionally, for SKINNY-128, we achieve effective 7-8 round neural distinguishers for the first time. Turning our attention to MIDORI-64 and MIDORI-128, we are successful in obtaining valid neural distinguishers for 3-4 rounds. The new results for the neural distinguishers for SKINNY and MIDORI are presented in Table 1.

The code related to our experiment has been uploaded to the GitHub repository, <https://github.com/CangXiXi/DL-based-SPN-Neural-Distinguishers-Framework>.

The rest of the paper is organized as follows. Section 2 gives a brief description of SPN block ciphers and introduces differential-neural cryptanalysis. In Sect. 3, we present an optimization framework for training neural distinguishers based on SPN block ciphers from training data format and neural network architecture. In Sect. 4, we validate the advantages of the improved framework on the SKINNY family and applied it to the MIDORI family. Finally, our work is summarized in Sect. 5.

## 2. Preliminaries

### 2.1 Notations

In this paper,  $K$  denotes the master key and  $k_i$  denotes the  $i$ -round subkey. We will call  $\Delta_{in}$  the input difference. Furthermore,  $(P, P')$  denotes the plaintext pair and  $(C, C')$  denotes the ciphertext pair. At last,  $\Delta C = C \oplus C'$  represent ciphertext difference.

### 2.2 SPN Block Ciphers

The Substitution Permutation Network (SPN) is a cipher design that employs series of mathematical operations, together satisfying confusion and diffusion properties defined by Shannon [1]. The class of SPN block cipher considered in this paper is described below. Its block length

is  $n$  bits (or  $n/d$ -word with a word being  $d$ -bit) and the round function consists of three basic operations: a non-linear substitution layer, a linear permutation layer and a round key addition layer. (1) The substitution layer can be partitioned into  $n$  parallel non-linear substitution boxes  $F_{2^d}^n \rightarrow F_{2^d}^n : S(x_1, x_2, \dots, x_n) = (s_1(x_1), s_2(x_2), \dots, s_n(x_n))$ , where each  $s_i$  is a non-linear bijective mapping on  $F_{2^d}^n$ . (2) The permutation layer is an invertible linear transformation  $P$ , which is commonly represented by some diffusion matrix or bit permutation, defined over  $F_{2^d}^{n \times n}$ . (3) The round key addition layer is defined simply by the XOR of the round subkey  $k_i$  and the input  $x$ .

### 2.3 A Brief Description of Two SPN Block Ciphers

We choose two SPN block ciphers for supporting our work.

#### 2.3.1 Brief Description of SKINNY

SKINNY is a family of lightweight tweakable block ciphers proposed by Beierle et al. at CRYPTO 2016 [2]. Let  $n$  denote the block size,  $t$  denote the tweak size, and  $c$  denote the cell size. The family of SKINNY has six main versions, SKINNY- $n/t$ : for each  $n \in \{64, 128\}$ , there are three tweak size versions:  $t = n$ ,  $t = 2n$ , and  $t = 3n$ . The round function of SKINNY is inspired by the Advanced Encryption Standard (AES), which follows a classical SPN structure. The state is updated in five operations: SubCells (SC), AddConstants (AC), AddRoundTweakey (ART), ShiftRows (SR), and MixColumns (MC). The matrix has a low weight, and the tweak is only XORed to the first two rows. This feature can be used to suggest a new sample data format for neural distinguishers. More details on SKINNY can be found in [2].

#### 2.3.2 Brief Description of MIDORI

MIDORI is a family of lightweight SPN block ciphers proposed by Banik et al. at ASIACRYPT 2016 [3]. Let  $n$  denote the block size. The family of MIDORI has two versions: MIDORI-64 and MIDORI-128, with  $n = 64$  and  $n = 128$ , respectively. The round function of MIDORI is a variant SPN structure. Each round function of MIDORI consists of the following transformations: AddRoundKey (ARK), SubCell (SC), ShuffleCell (ShC), and MixColumns (MC). More details on MIDORI can be found in [3].

### 2.4 Differential-Based Neural Distinguishers

In the analysis of block cipher security, differential cryptanalysis [5] is one of the most effective attack methods. It is a chosen-plaintext attack and was first introduced by Biham and Shamir in 1990 to analyze the DES block cipher. Specifically, in a random permutation with a block size of  $2n$ , the average probability of a differential is:  $\Pr(\Delta_{in} \rightarrow \Delta_{out}) = 2^{-2n}$  for all  $\Delta_{in}, \Delta_{out}$ . If an attacker can find a difference such that  $\Pr(\Delta_{in} \rightarrow \Delta_{out}) > 2^{-2n}$ , then this difference is called

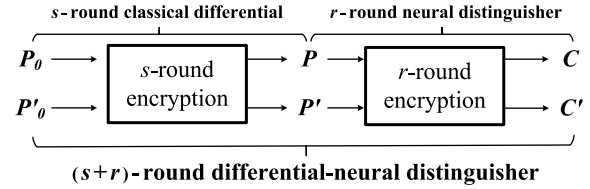


Fig. 1 The framework of differential-neural distinguisher

a differential distinguisher and can be used to attack cipher algorithms.

In [9], Gohr proposed a new method of differential cryptanalysis combined with deep learning. This deep learning-based differential cryptanalysis can be divided into two parts, according to the traditional differential cryptanalysis: generation of neural distinguishers and subkey recovery based on neural distinguishers. The generation of neural distinguishers consists of two steps: training data generation and neural network training.

#### Step1: Training data generation

$10^7$  plaintext pairs  $(P, P')$  are randomly generated with a fixed difference  $\Delta_{in}$ .  $10^7$  labels  $Y \in \{0, 1\}$  are randomly generated and assigned to the samples. For the samples with label 1, which are encrypted from plaintext pairs through  $r$  rounds to generate ciphertext pairs; for the samples with label 0, re-generate the right half of plaintext pairs  $P'$  randomly, and then encrypt  $r$  rounds to generate ciphertext pairs.

#### Step2: Neural network training

Define the structure of the residual neural network used for training. Based on the training samples generated above, the neural network is trained to perform a binary classification task: distinguish between the ciphertext pairs encrypted with fixed differences and the random ciphertext pairs. The trained neural network is evaluated by generating  $10^6$  test samples (repeating Step 1), and if the accuracy is higher than 50%, the network is considered a valid neural distinguisher.

Then, Gohr can extend the distinguisher by adding classical differential transformations in front of the above neural distinguisher. We call the extended distinguisher as differential-neural distinguishers, and its framework composition is shown in Fig. 1.

## 3. Description of Improved Framework for SPN Neural Distinguishers

### 3.1 Motivation

In [9], Gohr showed that neural networks can be trained as differential-neural distinguishers, capable of distinguishing between ciphertext pairs with fixed input differences and ciphertext pairs with random input differences. The details of this technology have been described in Sect. 2.4. Subsequently, many researchers have customized the neural distinguisher for various cipher algorithms, such as SIMON distinguishers [13], SIMECK distinguishers [15], and KATAN distinguishers [21], etc. By summarizing the existing works, we find that the current approaches suffer from the following

shortcomings.

- The training data format needs to be standardized from the practical application perspective. The current training data format is limited to the optimization of a single cipher algorithm, which is not universal and cannot form a common training data format framework for a certain type of cipher structure (such as Feistel, SPN, etc.).
- Lacking breakthroughs in neural network architecture from a deep learning perspective. The neural network construction technology is getting better and better in computer vision. However, the current neural network architecture used for neural distinguishers training is single, and some reasonable and advanced technologies need to be applied.

Firstly, in computer vision, the modification of training samples is a well-established technique known as data augmentation, which is commonly used to ensure that a sufficient amount of training data is available. In the training of neural distinguishers, the training samples are derived from encrypted randomly generated ciphertext pairs, thus guaranteeing the number. However, unlike image data, the confusion and diffusion properties of the ciphertext pairs after multiple rounds of encryption result in the weakening of the learnable features of the input difference, making it difficult for the neural network to extract features. Therefore, we decided to explore data augmentation techniques in computer vision to strengthen the ciphertext pairs training samples. In practical application, to make the data augmentation technique more universal, we target a class of cipher algorithms. There are mainly two types of foundational structures, the Feistel structure, and the SPN structure. Since many Feistel structure algorithms already have exclusive data augmentations, we focus on SPN cipher algorithms.

Secondly, during the training of deep neural networks, the initial step involves the construction of the neural network architecture. In the deep learning image recognition task, due to the limited ability of one-dimensional convolution (Conv1D) to process the data in matrix form, Conv2D is chosen to process the data for enhancing spatial feature learning ability. Therefore, we will explore the effect that the neural network constructed by Conv2D can achieve in the SPN neural distinguishers. Meanwhile, the choice of neural network hyperparameters has an impact on the training results. An SPN block cipher usually has different group sizes, so we use automatic optimization techniques to guide the selection of hyperparameters for large-state SPN block cipher neural distinguishers.

### 3.2 Generation of the Datasets

Differentiating between ciphertext pairs with input difference  $\Delta_{in}$  and randomly generated ciphertext pairs can be transformed into a binary classification problem, which is ideally suited for neural distinguishers. At EUROCRYPT 2021, Benamira et al. [22] proposed that the source of features used

to make distinctions during neural network training comes from the ciphertext difference and the internal state difference in penultimate rounds. At the same time, for the neural distinguishers of one cipher algorithm, their accuracy decreases with the increase of the number of rounds. Based on the above conclusions, we generalise the following generic data augmentation strategy under the conditions of known cipher algorithm round function components and unknown encryption master key  $K$  and subkey  $k_i$  for improving neural distinguisher training:

*Providing the training data format as close as possible to the previous round state for neural network can help to enhance the capability of the neural distinguishers.*

The above strategy is applicable to all cipher algorithms with improved neural distinguishers training data format, and we apply it to SPN structures in particular. Observing the round function construction of SPN block ciphers, the ciphertext can obtain the correct intermediate state value by inverse operation. Therefore, we classify the round function structures of SPN block ciphers which satisfy different conditions to set specific data augmentation methods. We will elaborate on the generation of data sets for training neural distinguishers. Considering a cipher  $E$  and a plaintext difference  $\Delta_{in}$ , the new data generation model for positive samples consists of three steps.

#### Step1: Training data generation

$10^7$  plaintext pairs  $(P, P')$  are randomly generated with a fixed difference  $\Delta_{in}$ .  $10^7$  labels  $Y \in \{0, 1\}$  are randomly generated and assigned to the samples. For the samples with label 1, which are encrypted from plaintext pairs through  $r$  rounds to generate ciphertext pairs; for the samples with label 0, re-generate the right half of the plaintext pairs  $P'$  randomly, and then encrypt  $r$  rounds to generate ciphertext pairs.

#### Step 2. Data augmentation techniques

The typical composition of the round function in SPN cipher algorithms includes both linear and nonlinear components, along with round key addition operation. Our data augmentation strategy is categorized into three cases based on the placement of round key addition operation within the round function.

*Case 1. Round key addition is not the final operation, and subkeys interact with selective state bits.*

Perform the inverse operation to the round key addition for ciphertext pairs  $(C, C')$  using inverse permutation operations like InvShiftRow and InvMixColumn. Continue the inverse substitution operation for the state bits not affected by the round key addition operation, all the way to the start of the round encryption. Record the state value resulting from this inverse operation on the ciphertext pair as  $(\tilde{C}, \tilde{C}')$ . Note the difference between the inverse pair of state values as  $\Delta\tilde{C}$ . The process is shown in Fig. 2.

*Case 2. Round key addition is not the final operation, and subkeys interact with all state bits.*

Perform inverse permutation operation for ciphertext pairs  $(C, C')$  up to the round key addition operation. Record the state value resulting from this inverse operation on the



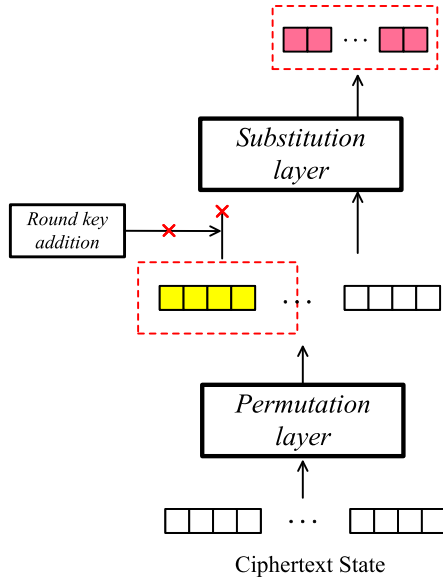


Fig. 2 Case 1-based SKINNY-64 data augmentation process

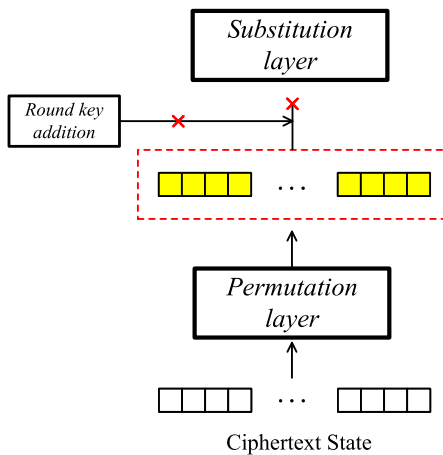


Fig. 3 Case 2-based SKINNY-64 data augmentation process

ciphertext pair as  $(\bar{C}, \bar{C}')$ . Note the difference between the inverse pair of state values as  $\Delta\bar{C}$ . The process shown in Fig. 3.

*Case 3. Round key addition is the final operation, and subkeys interact with all state bits.*

The subkey is unknown, implementing the data augmentation idea described above is not feasible in this case. Since applying the subkey is an Exclusive-OR operation in SPN block ciphers, the ciphertext difference is not affected by round key addition operation. Therefore, the ciphertext difference  $\Delta C$  can be calculated by inverse permutation operation such as InvShiftRow and InvMixColumn. Record the state value resulting from this inverse operation on the ciphertext difference as  $\Delta\bar{C}$  and  $(\bar{C}, \bar{C}')$  equals  $(C, C')$ . The process of data augmentation is shown in Fig. 4.

### Step 3. Training sample generation

In the final step, we combine  $(\bar{C}, \bar{C}')$  with  $\Delta\bar{C}$  to create the training sample  $(\bar{C}, \bar{C}', \Delta\bar{C})$ .

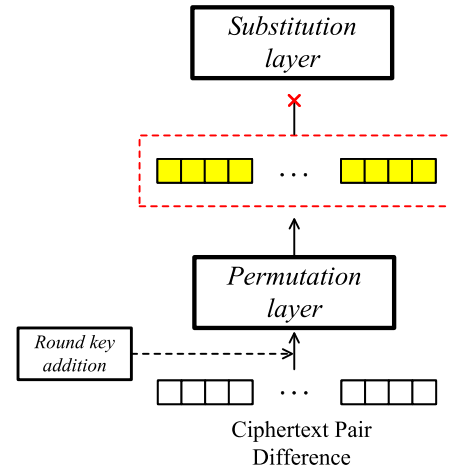


Fig. 4 Case 3-based SKINNY-64 data augmentation process

The above steps apply to the generation of positive samples. For negative samples, we also follow the above steps to generate, with the only difference being that the input difference  $\Delta_{in}$  used for each training sample is random. Compared to common data formats, our data format has the following advantages:

**Adaptability:** Our proposed training data format demonstrates remarkable flexibility, accommodating a wide range of scenarios within SPN cipher algorithms. It systematically considers diverse interactions between subkeys and state bits, thus bolstering training robustness and rendering it suitable for diverse cipher configurations and key scenarios.

**Informativeness:** The results of Benamira et al. [22] emphasize the importance of the penultimate round in Gohr's neural distinguisher. Our proposed data augmentation is based on this conclusion. Taking into account the characteristics of SPN block ciphers' design and providing the neural network with information as close as possible to the state of the penultimate round.

**Future-Ready:** The novel input data format pioneers a promising avenue for data format generation in neural distinguishers. It provides improved ideas for neural distinguishers of all block ciphers and ample scope for improvements in the ongoing development of neural differential cryptanalysis.

### 3.3 New Network Structure

A neural network, often regarded as the fundamental component of a deep learning architecture, is a hierarchical arrangement of interconnected modules, most of which are subject to learning. These modules are designed to compute non-linear input-output transformations. Each module within this architectural stack is tasked with transforming its input data, progressively increasing both selectivity and invariance within the feature representation.

In computer vision, the architecture of neural networks is based on image samples, typically characterised by their dimensions (width and height) and the number of channels (usually 1 for grayscale images and 3 for colour images).

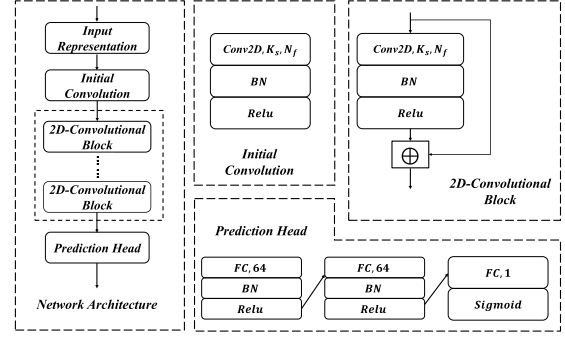
Training samples for neural distinguishers typically consist of ciphertext pairs from the target cipher. AES-like SPN block ciphers often represent their state values in matrix form. Consequently, we find similarities between the training samples of ciphertext pairs and colour images in the data representation (for a single channel image sample and a single ciphertext, both can be represented as two-dimensional data). These similarities motivate us to integrate a useful technology from computer vision into the neural distinguishers. Conv2D is effective in extracting various image features, including edges, textures, shapes and colours and also handles multi-channel color images well. By replacing Conv1D with Conv2D, our network exhibits increased sensitivity to spatial relationships within the data, allowing it to decipher fine-grained details and features that may be critical for accurate detection and classification of ciphertext pairs. For non-AES-like SPN block ciphers, the Conv2D neural network structure remains applicable, albeit with adjustments required for the dimensions of the input cipher pairs.

To illustrate the profound implications of this architectural extension, we have employed a ResNet structure similar to that presented in Gohr’s work [9] as our foundation. The ResNet structure main components are four parts: an input layer for processing training datasets, an initial convolutional layer, a residual tower consisting of multiple Conv2D, and a prediction head consisting of fully connected layers.

**Input representation.** We use the new data format on the input format to demonstrate the excellence of two-dimensional in three-dimensional training data. The neural network accepts data of the form  $(\bar{C}, \bar{C}', \Delta\bar{C})$ , arranged in a  $3 \times 4 \times n/4$  array.  $n$  is the block size of the target cipher. (For SKINNY-64, the block size  $n = 64$ ).

**Initial convolution.** Building upon Gohr’s foundational work [9], the input layer feeds into an optimized initial convolution layer structured as Conv2D-Batch Normalization-ReLU. The Conv2D layer, configured with  $2 \times 2$  kernels, processes the  $3 \times 4 \times n/4$  input tensor to model spatial diffusion patterns inherent in SPN state matrices, directly capturing cross-byte relationships induced by permutation layers. Batch normalization stabilizes training by normalizing activations, reducing internal covariate shift and mitigating gradient vanishing/explosion. ReLU activation introduces efficient nonlinearity, enabling faster SGD convergence. This adaptation enhances feature extraction for SPN ciphers.

**Conv2D blocks.** The initial convolution is associated with the Conv2D blocks. The difference between the Conv2D blocks and the initial convolution block lies in the adoption of residual structure. The residual connection can directly transfer shallow features to deep layers, thereby improving the gradient flow efficiency. Residual connections enable direct transmission of shallow layer features to deeper layers, improving gradient flow via an “identity mapping” mechanism, mathematically expressed as:  $y_l = x_l + F(x_l, W_l)$ , where  $x_l$  and  $y_l$  denote layer inputs/outputs, and  $F$  represents the residual mapping. The structure preserves shallow-layer features while learning deep abstractions, ensuring consistent feature representation



- **Conv2D:** Two-dimensional convolutional layer, utilized to model the matrix-structured state of SPN ciphers.
- **Ks:** Kernel size of the Conv2D layer.
- **Nf:** Number filters in the Conv2D layer.
- **FC:** Fully connected layer, which aggregates high-dimensional features into a lower-dimensional semantic space. In the prediction head, (FC,64) denotes a 64-unit dense layer for feature integration, and (FC,1) is the final single-unit layer for binary classification.

**Fig. 5** New network structure

across layers. This is critical for modeling SPN confusion-diffusion properties in cryptographic analysis.

**Prediction head.** The Conv2D blocks are connected to the prediction head. The first two layers are densely connected layers of 64 units, followed by batch normalization and a ReLU activation function. Then the last layer consists of a single output unit using the Sigmoid activation function to output a binary classification result. Design logic of the first two layers: The two fully connected layers with 64 units strike a balance between computational efficiency and feature expressiveness. This dimensionality is sufficient to capture critical cryptographic features while mitigating overfitting risks. By BN and ReLU activation, we standardize the input distribution to reduce internal covariate shift and introduce nonlinear transformations to model complex relationships within the 64-dimensional feature space, ensuring robust and stable feature extraction; Design logic of the final layer: The single-output unit with a Sigmoid activation function maps the 64-dimensional feature vector directly to the  $[0, 1]$  probability interval, precisely tailored to the binary classification requirements of cryptographic distinguishers (distinguishing between real cipher structures and random permutations).

We refer to Fig. 5 for a description of the network architecture. Once the overall structure of the neural network has been determined, the critical consideration shifts to the selection of hyperparameters. The proper hyperparameter choices can lead to a substantial enhancement in neural network performance. Effectively controlling network complexity and mitigating the risk of overfitting can be achieved through the proper configuration of regularization parameters (e.g., L1 or L2 regularization) and dropout rates. The hyperparameter search space was initially established by referencing the foundational work of Gohr [9]. Specifically, we leveraged the parameter configurations proposed in Gohr’s research as a baseline to define the initial search ranges for critical hyper-

**Table 2** Parameters of the network architecture for training neural distinguishers

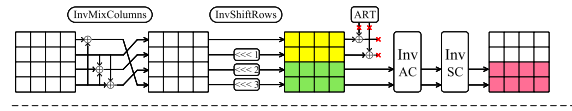
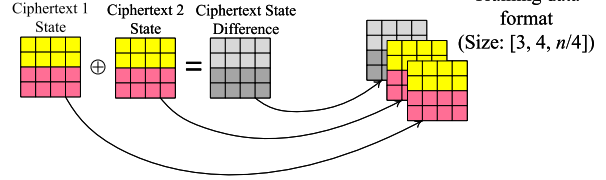
| Hyperparameters      | Value         |               |
|----------------------|---------------|---------------|
|                      | Small - state | Large - state |
| Train size           | $10^7$        | $10^7$        |
| Validation size      | $10^6$        | $10^6$        |
| Input representation | $[2, 4, n/4]$ | $[2, 4, n/4]$ |
| Depth                | 5             | 10            |
| Epochs               | 50            | 80            |
| Batch size           | 5000          | 10000         |
| Neurons              | 32            | 64            |
| Kernel size( $k_s$ ) | 3             | 3             |
| Number filters       | 32            | 64            |
| Optimizer            | Adam          | Adam          |
| Loss function        | MSE           | MSE           |

parameters. The computing environment for neural network hyperparameter search and neural distinguisher training in this paper is as follows: The computing environment consists of a hardware setup featuring an NVIDIA RTX 3080 with 10 GB VRAM, an Intel Core i7-11700K CPU (8 cores, 3.6 GHz), and 64 GB of DDR4 RAM. Using Holdout Validation, the hyperparameter search time was calculated by evaluating each parameter range with a single training-testing split and only used 30 epochs to observe the convergence speed. The small-state model (3–12 layers) took approximately 7.33 h for network depth, while the large-state model (8–15 layers) required around 11.67 h, totaling roughly 19 h. Batch size searches for small-state (2000–6000) and large-state (8000–12000) took about 3.67 h and 4.38 h respectively, summing to approximately 8.04 h. The exploration of the number of neurons (16, 32, 64) took approximately 2.2 h for the small-state model and 4.38 h for the large-state model, totaling about 6.58 h. Similarly, the search for kernel sizes ( $3 \times 3$ – $5 \times 5$ ) took approximately 6.58 h and the search for optimizers (Adam, SGD, RMSprop) also took approximately 6.58 h. Cumulatively, the entire hyperparameter search took roughly 46.49 h. The final hyperparameter configurations for the neural network are presented in Table 2.

In the field of binary classification, where the neural distinguishers play a central role, it is imperative to establish an appropriate performance metric. The metric chosen in this paper is accuracy, represented as the ratio of correctly classified samples to the total number of samples in the dataset, serves as a reliable measure of the capability of the neural distinguishers. Theoretically, if the accuracy of such a classifier exceeds the 50% threshold, it means more than just a random classification. Instead, it indicates an ability to distinguish beyond random guessing. In other words, an accuracy score above 50% indicates that the neural distinguishers has the acumen to effectively distinguish, making it a valuable tool in practical differential cryptanalysis.

#### 4. Advantage Verification and Application

In this section, we focus on the application of our neural

**Step 1. Ciphertext state change****Step 2. Data format composition****Fig. 6** Case 1-based SKINNY-64 data augmentation process

distinguisher training model to two common SPN block ciphers, SKINNY and MIDORI. When applied to SKINNY, we first evaluate the advantages of the data augmentation and the neural network structure separately. The validation results are presented concurrently with the final distinguisher results for SKINNY in this section. We then extend the improvement approach to different versions of MIDORI with varying block lengths, demonstrating the broad applicability of the method.

##### 4.1 Advantage Verification on SKINNY

In this subsection, we validate the advantages of the proposed training data format and neural network structure. For this purpose, we need to select the target algorithm for the neural distinguishers. Due to the relationships between the different cases that are included, if a cipher algorithm's round function structure corresponds to Case 1, it can be performed with both Case 2 and Case 3. Among the SPN block ciphers, we chose SKINNY as the target algorithm.

First, a detailed description of the SKINNY application data augmentation process. The specific operation can be divided into two steps, which are also shown in the Fig. 6.

##### *Case 1 (SKINNY-64 Original Structure)*

- **Inverse Permutation:** Apply InvShiftRows and InvMixColumns to ciphertext pairs  $(C, C')$  to reverse the permutation layer, undoing the linear diffusion of the round function.
- **Selective Inverse Substitution:** Perform inverse substitution on state bits unaffected by subkeys (SKINNY's rows 3–4), stopping at the key addition layer for rows 1–2 (where subkeys interact selectively).
- **Data Formatting:** Combine inverse states  $(\bar{C}, \bar{C}')$  and difference  $\Delta \bar{C}$  into a 3D tensor  $[3, 4, 16]$ , leveraging the  $4 \times 16$  state matrix for Conv2D feature extraction.

Since our proposed model suggests improvements in the training data format and neural network structure respectively, we adopt a step-by-step approach to verify the advantages of the two techniques. In [23], Gohr has trained an optimized version of the SKINNY-64 neural distinguishers. We set up the same neural network structure and input difference (0x0000 0000 0000 2000), so the difference between

**Table 3** Summary of the neural distinguishers for SKINNY family

| Cipher     | Round          | Pair  |       |        |        |
|------------|----------------|-------|-------|--------|--------|
|            |                | 1     | 2     | 4      | 8      |
| SKINNY-64  | 7 <sup>1</sup> | 93.7% | 99.2% | 100.0% | 100.0% |
|            | 7              | 97.6% | 99.9% | 100.0% | 100.0% |
|            | 7              | 99.2% | 99.9% | 100.0% | 100.0% |
|            | 8              | 54.1% | 58.4% | 64.6%  | 71.5%  |
| SKINNY-128 | 7              | 99.7% | 99.9% | 100.0% | 100.0% |
|            | 8              | 51.0% | 51.5% | 52.1%  | 53.1%  |

<sup>1</sup> The trail that used to derive this distinguisher comes from Gohr *et al.*'s work [23]. Others are newly found in our work.

the neural distinguishers is only the data format. If the new SKINNY-64 neural distinguishers achieve a breakthrough in accuracy and number of rounds, our proposed data augmentation for SPN block ciphers is advantageous. Next, with the new neural network structure (Fig. 5) based on the data augmentation, it is possible to determine the improvement that can be achieved by Conv2D. The experimental results are shown in Table 3.

Under the assumption of statistical independence, the multi-sample approach aggregates predictions from  $n$  samples with the same label. Specifically, for  $n$  samples with the same label, the base neural distinguisher independently predicts probabilities  $z_{\text{atomic}}$  for each sample. The final decision is derived through likelihood ratio calculation: First, compute the joint probability for the encrypted case

$$\text{LR}_{\text{enc}} = \prod_{i=1}^n z_{\text{atomic}}[i],$$

representing the probability that all samples are encrypted pairs. Simultaneously, compute the joint probability for the random case

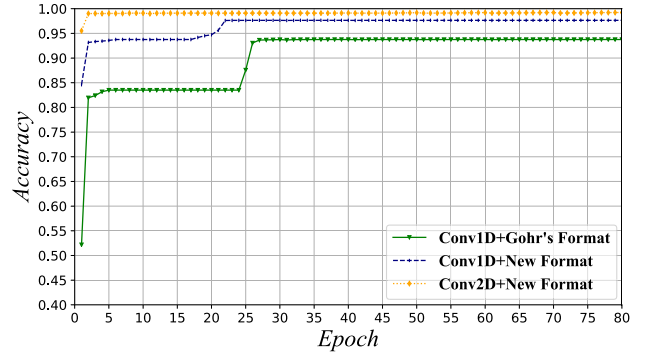
$$\text{LR}_{\text{rnd}} = \prod_{i=1}^n (1 - z_{\text{atomic}}[i]),$$

denoting the probability that all samples are random pairs. The likelihood ratio  $\frac{\text{LR}_{\text{rnd}}}{\text{LR}_{\text{enc}}}$  is then normalized to obtain the final prediction score

$$z = \frac{1}{1 + \frac{\text{LR}_{\text{rnd}}}{\text{LR}_{\text{enc}}}},$$

which simplifies to  $z = 1$  if  $\text{LR}_{\text{enc}} > \text{LR}_{\text{rnd}}$  (indicating encryption) and  $z = 0$  otherwise. In edge cases where numerical instability occurs (e.g.,  $\text{LR}_{\text{enc}} \approx 0$ ), the decision defaults to majority voting among individual samples to ensure robustness.

The subsequent analyses are based on this 1-pair neural distinguisher. It is important to note that if the training of 1-pair sample yields excellent results, this excellence is maintained even when extended to multiple ciphertext pair samples. Upon examination of the results presented in Table 3, the 7-round SKINNY-64 neural distinguisher achieved an accuracy of 97.6% when using the same neural network

**Fig. 7** 7-round SKINNY-64 verification accuracy

structure and hyperparameter settings with different data formats. This indicates an improvement over the existing best SKINNY-64 result (93.7%), confirming the effectiveness of the data format proposed in this paper. Then, we examined the training results using both the new data format and the new neural network structure using Conv2D. This transition from Conv1D to Conv2D consistently improved the accuracy of the 7-round SKINNY-64 neural distinguishers (99.2%).

In particular, the training model led to the successful development of an effective 8-round SKINNY-64 neural distinguisher for the first time, achieving an accuracy of 54.1%. In summary, the advantages of both the new data format and the network structure are verified. Thus, the SPN block ciphers neural distinguisher framework is advantageous.

The training of the large-state SKINNY-128 used improved data formatting and neural network architecture. In addition, hyperparameter auto-optimization techniques were used in the final training process. Based on the results in Table 3, the 7-round SKINNY-128 neural distinguishers achieved the accuracy of 99.7%. Furthermore, the training produced an effective 8-round SKINNY-128 neural distinguisher that achieved the accuracy of 51.0%.

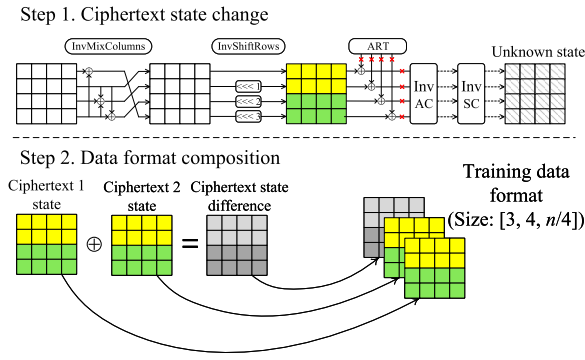
Furthermore, Fig. 7 shows the variation in validation set accuracy under three different training conditions. It can be seen that the neural network training converges consistently in all three cases, without accuracy fluctuations in accuracy after convergence. The Conv1D neural network architecture achieves convergence between 20 and 30 epochs, while the Conv2D neural network achieves convergence within the first 5 epochs. This underlines the superior learning capability and efficiency of Conv2D over Conv1D, and thus explains the rationale behind the neural network structure proposed in this study.

Considering that the SKINNY algorithm belongs to Case 1, it can be modified to meet the criteria of Case 2 and Case 3. In Case 2, we assume a SKINNY variant with global key addition, and the steps for describing the training data format are as follows, which are also shown in the Fig. 8.

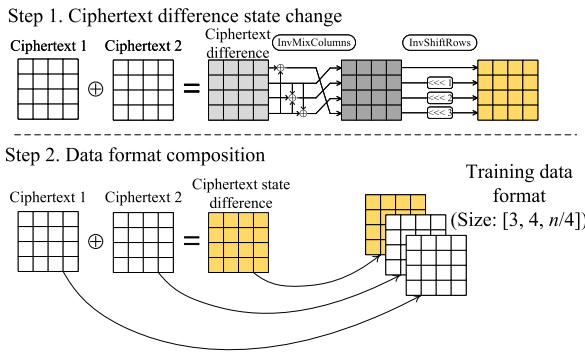
#### Case 2 (Hypothetical SKINNY Variant with Global Key Addition)

- Inverse Permutation: Apply InvShiftRows and InvMixColumns to  $(C, C')$  to reverse permutation, similar to





**Fig. 8** Case 2-based SKINNY-64 data augmentation process



**Fig. 9** Case 3-based SKINNY-64 data augmentation process

#### Case 1.

- Global Inverse Substitution: Perform inverse substitution on all state bits (rows 1–4), as subkeys in this variant interact with all bits before permutation.
- Data Formatting: Construct 3D tensor  $[3, 4, 16]$  from full inverse states and  $\Delta\tilde{C}$ , enabling Conv2D to model cross-byte dependencies without selective key stopping points.

In Case 3, we assume a SKINNY variant with end-of-round key addition. The core feature of this variant is that the pre-permutation difference state can be directly derived by reversing the permutation layer without performing inverse substitution, and the specific operation steps are shown in the process of Case 3, which are also shown in the Fig. 9.

#### Case 3 (SKINNY Variant with End-of-Round Key Addition)

- Difference Reversal: Since key addition is final, apply InvShiftRows and InvMixColumns to ciphertext difference  $\Delta C$  to derive pre-permutation difference  $\Delta\tilde{C}$ .
- Direct Data Formatting: Use original ciphertext pairs  $(C, C')$  and  $\Delta\tilde{C}$  as inputs, as key addition at round end makes inverse substitution unnecessary for bias-free augmentation.

The results are shown in the Table 4. In particular, the accuracy of the neural distinguishers gradually decreases from Case 1 to Case 3, which is consistent with what we expected when we initially configured these cases. The results illustrate the importance of the component arrangement

**Table 4** The neural distinguishers Accuracy Results of Different Cases for 7-round SKINNY-64

| Cipher    | Case   | Accuracy |
|-----------|--------|----------|
| SKINNY-64 | Case 1 | 99.2%    |
|           | Case 2 | 97.4%    |
|           | Case 3 | 93.8%    |

**Table 5** Summary of the neural distinguishers for MIDORI family

| Cipher     | Round | Pair  |       |       |       |
|------------|-------|-------|-------|-------|-------|
|            |       | 1     | 2     | 4     | 8     |
| MIDORI-64  | 3     | 99.3% | 99.9% | 100%  | 100%  |
|            | 4     | 86.1% | 95.0% | 99.3% | 99.9% |
| MIDORI-128 | 3     | 97.2% | 98.8% | 99.9% | 100%  |
|            | 4     | 72.8% | 80.5% | 88.6% | 95.5% |

and the round key addition setup in the design phase of SPN block ciphers.

## 4.2 Application to MIDORI

The new training model can also be effectively applied to MIDORI. First, since the AddRoundKey is the last operation in the round function of MIDORI, the Case 3 data augmentation process is employed. The input differences used in the MIDORI-64 and MIDORI-128 are  $(0x0000\ 0000\ 0000\ 0002)$  and  $(0x00000000\ 00000000\ 00000000\ 00000002)$ , respectively. For MIDORI-64, the accuracy achieved by 3 rounds of neural distinguisher training is 99.3%, while for 4 rounds of neural distinguisher it is 86.1%. For MIDORI-128, the accuracy achieved by training a 3-round neural distinguisher is 97.2%, while for a 4-round neural distinguisher the accuracy is 72.8%. Under the independence assumption, the application of the trained 1-pair neural distinguishers using the idea of multiple pairs all showed a significant increase in accuracy. Detailed results are shown in Table 5.

## 4.3 Training Time

For training, the neural distinguishers for each 64-bit block size SKINNY-64 and MIDORI-64 took approximately 1.2 h, while those for each 128-bit block size SKINNY-128 and MIDORI-128 required about 3.9 h.

## 5. Conclusion

In this paper, we propose a deep learning-based neural distinguisher framework specifically for SPN block ciphers from two aspects. Firstly, we classify the SPN structure into three cases based on the component arrangement and the round key addition setup, and propose three data augmentation approaches applied to improve the training sample format, respectively. Compared with the existing training sample format, our data augmentation enriches the learnable features in the sample data, which contributes to the accuracy improvement. Secondly, considering that the permutation layer significantly mobilizes the bit positions, we vary the

training data to a three-dimensional size and switch to a Conv2D for feature extraction in the neural network. Compared with the original neural network structure, Conv2D enhances spatial feature learning capability. In addition, we provide optimized hyperparameters setting options for large-state SPN block ciphers. Together, the above techniques form our neural distinguisher framework for SPN block ciphers. To demonstrate the effectiveness of the new framework, it is applied to two typical SPN block cipher algorithms: SKINNY and MIDORI, achieving the best neural distinguisher known so far.

## References

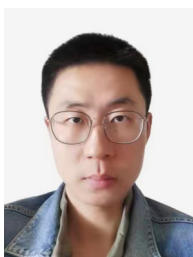
- [1] C.E. Shannon, "Communication theory of secrecy systems," *The Bell system technical journal*, vol.28, no.4, pp.656–715, 1949.
- [2] C. Beierle, J. Jean, S. Kölbl, G. Leander, A. Moradi, T. Peyrin, Y. Sasaki, P. Sasdrich, and S.M. Sim, "The SKINNY family of block ciphers and its low-latency variant MANTIS," *Advances in Cryptology – CRYPTO 2016*, Berlin, Heidelberg: Springer, vol.9815, pp.123–153, 2016.
- [3] S. Banik, A. Bogdanov, T. Isobe, K. Shibutani, H. Hiwatari, T. Akishita, and F. Regazzoni, "Midori: A block cipher for low energy," *Advances in Cryptology – ASIACRYPT 2015*, Berlin, Heidelberg: Springer, vol.9453, pp.411–436, 2015.
- [4] A. Bogdanov, L.R. Knudsen, G. Leander, C. Paar, A. Poschmann, M.J.B. Robshaw, Y. Seurin, and C. Vikkelsoe, "PRESENT: An ultra - lightweight block cipher," *Cryptographic Hardware and Embedded Systems CHES 2007*, Berlin, Heidelberg: Springer, vol.4727, pp.450–466, 2007.
- [5] E. Biham and A. Shamir, "Differential cryptanalysis of DES-like cryptosystems," *Journal of CRYPTOLOGY*, vol.4, no.1, pp.3–72, 1991.
- [6] M. Matsui, "Linear cryptanalysis method for DES cipher," *Workshop on the Theory and Application of Cryptographic Technique 1993*, Berlin, Heidelberg: Springer, vol.765, pp.386–397, 1993.
- [7] L. Knudsen and D. Wagner, "Integral cryptanalysis," *FSE 2002*, Berlin, Heidelberg: Springer, vol.2365, pp.112–127, 2002.
- [8] A. Bogdanov and V. Rijmen, "Linear hulls with correlation zero and linear cryptanalysis of block ciphers," *Designs, codes and cryptography*, vol.70, no.3, pp.369–383, 2014.
- [9] A. Gohr, "Improving Attacks on Round - Reduced Speck32/64 Using Deep Learning," *Advances in Cryptology – CRYPTO 2019*, Berlin, Heidelberg: Springer, vol.11693, pp.150–179, 2019.
- [10] Z. Bao, J. Guo, M. Liu, L. Ma, and Y. Tu, "Enhancing differential-neural cryptanalysis," *Advances in Cryptology – ASIACRYPT 2022*, Berlin, Heidelberg: Springer, vol.13791, pp.318–347, 2022.
- [11] N. Băcuieți, L. Batina, and S. Picek, "Deep Neural Networks Aiding Cryptanalysis: a Case Study of the Speck Distinguisher," *Applied Cryptography and Network Security*, Rome, Italy: Springer, vol.13269, pp.809–829, 2022.
- [12] Y. Chen, Y. Shen, H. Yu, and S. Yuan, "A new neural distinguisher considering features derived from multiple ciphertext pairs," *The Computer Journal*, vol.66, no.6, pp.1419–1433, 2023.
- [13] Z.Z. Hou, J.J. Ren, and S.Z. Chen, "Improve neural distinguishers of simon and speck," *Security and Communication Networks*, vol.2021, no.1, pp.1–11, 2021.
- [14] A. Ebrahimi, F. Regazzoni, and P. Palmieri, "Reducing the Cost of Machine Learning Differential Attacks Using Bit Selection and a Partial ML-Distinguisher," *International Symposium on Foundations and Practice of Security*, Cham: Springer Nature Switzerland, vol.13877, pp.123–141, 2022.
- [15] J. Lu, G. Liu, B. Sun, C. Li, and L. Liu, "Improved (related-key) differential-based neural distinguishers for SIMON and SIMECK block ciphers," *The Computer Journal*, vol.67, no.2, pp.537–547, 2024.
- [16] A. Hambitzer, D. Gerault, Y.J. Huang, N. Aaraj, and E. Bellini, "NNBits: Bit Profiling with a Deep Learning Ensemble Based Distinguisher," *ryptographers' Track at the RSA Conference*, San Francisco, CA, USA: Springer, vol.13871, pp.493–523, 2023.
- [17] X. Yue and W. Wu., "Improved neural differential distinguisher model for lightweight cipher speck," *Applied Sciences*, vol.13, no.12, p.6994, 2023.
- [18] G. Wang, G. Wang, and S. Sun, "A New (Related-Key) Neural Distinguisher Using Two Differences for Differential Cryptanalysis," *IET Information Security*, vol.2024, no.1, pp.1–11, 2024.
- [19] D. Shen, Y. Song, Y. Lu, S. Long, and S. Tian, "Neural differential distinguishers for GIFT-128 and ASCON," *Journal of Information Security and Applications*, vol.82, no.103758, 2024.
- [20] Y. Hu, L. Li, S. Zhu, and Z. Hu, "Enhancing neural distinguishers with partial difference bits leakage," *Internet of Things*, vol.29, no.101438, 2025.
- [21] D. Lin, M. Li, Z. Hou, and S. Chen, "Conditional differential analysis on the KATAN ciphers based on deep learning," *IET Information Security*, vol.17, no.3, pp.347–359, 2023.
- [22] A. Benamira, D. Gerault, T. Peyrin, and Q.Q. Tan, "A deeper look at machine learning-based cryptanalysis," *Advances in Cryptology – EUROCRYPT 2021*, Berlin, Heidelberg: Springer, vol.12696, pp.805–835, 2021.
- [23] A. Gohr, G. Leander, and P. Neumann, "An assessment of differential - neural distinguishers," *Cryptology ePrint Archive*, Paper 2022/1521, <https://eprint.iacr.org/2022/1521>, 2022.



**Jiashuo Liu** received the B.S. and M.S. degrees from Information Engineering University, Zhengzhou, China. He is currently pursuing the Ph.D. degree with Information Engineering University, Zhengzhou, China. His research interests include intelligent cryptanalysis and artificial intelligence security.



**Manman Li** received the B.S. degree from Xidian University, Xi'an, China, and the M.S. and Ph.D. degrees from Information Engineering University, Zhengzhou, China. She is currently a Lecturer with the School of Cyberspace Security, Information Engineering University. Her research interests include symmetric cryptography and intelligent cryptanalysis.



**Jiongjiong Ren** received the B.S., M.S., and Ph.D. degrees from Information Engineering University, Zhengzhou, China. He is currently a Lecturer with the School of Cyberspace Security, Information Engineering University. His research interests include symmetric cryptographic algorithm design and intelligent cryptanalysis.



**Shaozhen Chen** received the B.S. degree from Information Engineering University, Zhengzhou, China and the Ph.D. degree from Shandong University, Jinan, China. She is currently a Professor with the School of Cyberspace Security, Information Engineering University. Her research interests include secure implementation of cryptographic algorithms and symmetric cryptanalysis.